NewsBot Individual Reflective Journal

Emmanuel ABABIO ATTA NJEH | ITAI 2373 | October 17, 2026

My project and learning experience

I worked individually on a NewsBot prototype using the BBC News Classification dataset in Google Colab. The goal was to bring several NLP techniques together so the system could categorize articles, analyze sentiment and identify entities. I learned that understanding each technique separately was not enough. I had to understand how the notebook cells worked together.

The most challenging part was adapting the starter notebook to real data. Its sample articles included titles, dates and sources, but my BBC CSV contained article IDs, text and categories. I renamed the available columns and removed references to fields that were not there. Repeated title errors taught me to inspect the dataset first instead of assuming that a template matches it. Setting full_text equal to content also meant that the two sentiment analyses would return the same scores.

Debugging and combining the components

Missing NLTK data stopped tokenization and POS tagging. I added the required downloads and reran the affected cells. I learned that an error can happen because a required file is missing, even when the function is written correctly. The two resource fixes were:

```
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')
```

Another error appeared when Naive Bayes received negative sentiment values. I learned that different algorithms have different input requirements: Multinomial Naive Bayes needs nonnegative features. Using MinMaxScaler inside its pipeline addressed that requirement. I also learned why the scaler belongs inside the pipeline: during cross-validation, it should learn from each training portion without using the validation portion.

The last prediction error happened because the training and prediction inputs did not match. Training used 5,000 TF-IDF values and four sentiment values, but the template added seven dummy values for new articles. The corrected prediction code used the actual sentiment scores:

```
scores = self.analyze_sentiment(full_text)
sentiment_features = np.array([[scores['compound'],
   scores['pos'], scores['neu'], scores['neg']]])
features_combined = np.hstack([
   features.toarray(), sentiment_features])
```

I also changed the method call to classify_article(processed_text, full_text). Both sides then used 5,004 features in the same order. Seeing all three demonstration articles run successfully was an important milestone. It taught me that the features need to have the same meaning and order, as well as the same number of values.

What the results taught me

TF-IDF and classification were especially useful because they turned article text into category predictions. However, I learned that a high accuracy score needs a trustworthy evaluation process. The original best accuracy was 97.32%, but TF-IDF had been fitted before splitting the data. The review also found 50 repeated article texts. Because of these issues, I could not treat the first accuracy score as the final answer.

The separate revised experiment removed exact duplicates and fitted features within training folds. It achieved 95.49% accuracy and 95.35% macro F1 on 288 held-out articles. TF-IDF alone performed better in training cross-validation than the tested combination with sentiment and length features. This taught me not to assume that adding more features automatically helps. I would still need to test new articles to see how well the model works outside this dataset.

The demonstrations helped me see what the overall score missed. The Microsoft story could fit tech or business, and the climate article had no environment category available. Its politics probability was only 25.97%, and its slightly positive sentiment score did not fit the alarming wording well. NER also produced the incorrect organization span "Microsoft Acquires." I learned that code can run without errors and still give an answer that does not make sense.

Useful applications and ethical responsibility

NewsBot could help editors organize incoming stories and help analysts find articles mentioning companies or people. I would use it as an assistant with human review, especially for uncertain predictions. If the system extracts a name incorrectly, it could connect the wrong person or company to a story. A sentiment score also cannot tell me what the public thinks or prove bias. I would explain these limits and let users read the original article.

Future learning and my portfolio

Next, I would try new full-length articles, check more entity mistakes and work on topics the model does not recognize well. Topic modeling could help discover subjects beyond the five category labels. Models that consider more context could also help with sentiment that is difficult to understand from individual words. I would compare these changes against the simpler baseline before claiming an improvement.

This project gave me practice reading tracebacks, checking columns and matching features across a pipeline. I used the course template and AI assistance for debugging, additional Student Task code and writing support. Reviewing those additions matters because I need to explain what they do and what their results mean. For my portfolio, I would present the notebook, setup instructions, measured results and examples of both useful and uncertain predictions. I completed the project individually, so no group roles or collaboration activities apply.

Project repository: https://github.com/NjehAbabio/ITAI2373-NewsBot-Midterm

Sources: supplied ITAI 2373 starter notebook, saved project outputs and BBC News Classification on Kaggle.